Real-world (convert a real function to async) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `async_client_conversion_practice.py`: this is the exercise where an async prerequisite becomes a real, reusable function — the exact `ask()` shape gets imported into a Doc12 FastAPI route later, unchanged. **If not:** Doc12 would be the first place you ever had to write an async OpenAI call that's actually safe to reuse, with no smaller version to trust.

Basic Version

Approach 1 — the direct way

```
# async_client_conversion_practice.py
from openai import AsyncOpenAI
import asyncio

client = AsyncOpenAI()

async def ask(question):
    response = await client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": question}],
    )
    return response.choices[0].message.content

answer = asyncio.run(ask("What is 2 + 2?"))
print(answer)
```

Expected output:

```
2 + 2 equals 4.
```

The only real changes from the sync Doc04 starting point: `AsyncOpenAI` instead of `OpenAI`, `async def` on the function, `await` in front of the API call, and `asyncio.run(...)` instead of

calling `ask(...)` directly. Nothing about the model, the messages, or the return value changed at all.

This version works correctly. It's missing type hints and an `if __name__ == "__main__":` guard — both fine for a first working version.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — typed, with a `main()` and a `__main__` guard

```
# async_client_conversion_practice.py
from openai import AsyncOpenAI
import asyncio

client = AsyncOpenAI()


async def ask(question: str) -> str:
    response = await client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": question}],
    )
    return response.choices[0].message.content


def main() -> None:
    answer = asyncio.run(ask("What is 2 + 2?"))
    print(answer)


if __name__ == "__main__":
    main()
```

Expected output:

```
2 + 2 equals 4.
```

Difference from Basic: the `-> str` type hint on `ask` documents exactly what the coroutine resolves to once awaited, same as the sync version would have had. The `if __name__ == "__main__":` guard, calling a plain `main()` that itself calls `asyncio.run(...)`, means this file can be imported elsewhere (a test, or a FastAPI route in Doc12) and reuse `ask()` directly with its own `await`, without accidentally triggering a second, nested event loop the way calling `asyncio.run()` again from inside an already-running one would (this is the exact `RuntimeError` covered in `first_coroutine`'s Intermediate Hint).

Approach 2 — a client-level timeout, and the SDK's specific timeout error

Story: an `AsyncOpenAI()` call with no timeout can, in principle, wait forever if OpenAI's servers stop responding — and unlike a blocking `time.sleep()` mistake, this won't even look wrong from the outside; the request just never comes back. **If not:** one hung API call in Doc12's real FastAPI service would freeze the worker handling it indefinitely, with no error to point at.

```
# async_client_conversion_practice.py
from openai import AsyncOpenAI, APITimeoutError
import asyncio

client = AsyncOpenAI(timeout=30.0)


async def ask(question: str) -> str:
    response = await client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": question}],
    )
    return response.choices[0].message.content


async def main() -> None:
    try:
        answer = await ask("What is 2 + 2?")
        print(answer)
    except APITimeoutError:
        print("the API call took too long and was cancelled")


if __name__ == "__main__":
    asyncio.run(main())
```

Expected output (normal case):

```
2 + 2 equals 4.
```

Expected output if OpenAI's servers genuinely hang past 30 seconds:

```
the API call took too long and was cancelled
```

`timeout=30.0` on the client is a default every call made through it inherits — no individual caller has to remember to wrap their own call in `asyncio.wait_for(...)`. `APITimeoutError` is the SDK's specific exception for this — catching it instead of a bare `except Exception:` means a real

bug elsewhere in `ask()` (a typo in `model=`, say) still surfaces as its own clear error instead of being mistaken for a slow API call.

Approach 3 — used from inside an already-async caller (a FastAPI route), the way it'll actually be used in Doc12

Story: an `async def` route that's mid-`await` when the server process gets asked to shut down needs that `await` to actually be cancellable, and needs to never call its own `asyncio.run()` — the framework already owns the event loop. **If not:** importing `ask()` into a real Doc12 route would crash with `RuntimeError: asyncio.run() cannot be called from a running event loop`, exactly the mistake `first_coroutine`'s Intermediate Hint names.

```
# async_client_conversion_practice.py
from openai import AsyncOpenAI, APITimeoutError

client = AsyncOpenAI(timeout=30.0)


async def ask(question: str) -> str:
    response = await client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": question}],
    )
    return response.choices[0].message.content


# a stand-in for Doc12's FastAPI route -- no asyncio.run() anywhere in here,
# because the web framework already owns and is running the event loop
async def handle_question_request(question: str) -> dict:
    try:
        answer = await ask(question)
        return {"answer": answer}
    except APITimeoutError:
        return {
            "error": "The model took too long to respond. Please try again."
        }
```

Expected output, if you call `asyncio.run(handle_question_request("What is 2 + 2?"))` from a script to simulate what the framework would do:

```
{'answer': '2 + 2 equals 4.'}
```

Notice `ask()` itself is completely unchanged from Approach 1 — only the caller changed. `handle_question_request` never calls `asyncio.run()` because, in real Doc12 code, FastAPI

already started and owns the event loop before your route function is ever called; it just `await`s `ask()` directly and turns a timeout into a clean error response instead of a crashed request.

Approach 4 — `async with` for the client, `async for` for a streamed reply

Story: Doc04 streamed replies with a plain `for chunk in stream:`. With `AsyncOpenAI`, each next chunk arrives over the network, so getting it needs an `await` — that's what `async for` does. And a client holds open network connections, which should be closed when you're done — `async with` does that, the async twin of Doc04's `with open(...)`. **If not:** every MCP project (Projects 7, 8, 11, 12) opens its server connection with `async with ... as session:`, and Doc09 streams graphs with `async for ... in graph.astream(...)` — both would be new syntax with no smaller example behind them.

```
# async_client_conversion_practice.py
import asyncio

from openai import AsyncOpenAI


async def ask_streaming(client: AsyncOpenAI, question: str) -> str:
    stream = await client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": question}],
        stream=True,
    )
    full_reply = ""
    async for chunk in stream:
        piece = chunk.choices[0].delta.content
        if piece:
            print(piece, end="", flush=True)
            full_reply = full_reply + piece
    print()
    return full_reply


async def main() -> None:
    async with AsyncOpenAI(timeout=30.0) as client:
        await ask_streaming(client, "What is 2 + 2?")
    print("client closed:", client.is_closed())


if __name__ == "__main__":
    asyncio.run(main())
```

Expected output (the reply appears piece by piece):

```
2 + 2 equals 4.
client closed: True
```

- `async with X as name:` runs X's async "open" step, gives you `name`, and runs its async "close" step when the block ends — even if an error happens inside.
- `async for item in stream:` is a `for` loop where fetching each next item is awaited, so other tasks can run while a chunk is on its way.
- Both only work inside `async def`, like `await`.

Approach 5 — LangChain's async methods: `ainvoke`, `astream`, and `gather`

Story: Doc05 listed `.ainvoke()` as one of every chain's methods. It's the same chain, awaited: `ainvoke` for `invoke`, `astream` for `stream`. Because they're awaitable, `asyncio.gather` can run several chain calls at once. **If not:** an async LangChain call in later docs would look like a different library, instead of the same chain with an `a` in front.

```
# async_client_conversion_practice.py
import asyncio
import time

from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

prompt = ChatPromptTemplate.from_messages([
    ("system", "Answer in one short sentence."),
    ("user", "{question}"),
])
chain = prompt | ChatOpenAI(model="gpt-4o-mini") | StrOutputParser()


async def main() -> None:
    answer = await chain.ainvoke({"question": "What is 2 + 2?"})
    print(answer)

    async for piece in chain.astream({"question": "Name a primary colour."}):
        print(piece, end="", flush=True)
    print()

    start = time.perf_counter()
    answers = await asyncio.gather(
        chain.ainvoke({"question": "What is 2 + 2?"}),
        chain.ainvoke({"question": "What is the capital of France?"}),
        chain.ainvoke({"question": "Name a primary colour."}),
    )
```

```
    elapsed = time.perf_counter() - start
    print(f"{len(answers)} answers in {elapsed:.2f}s")


if __name__ == "__main__":
    asyncio.run(main())
```

Expected output (wording and time vary):

```
2 + 2 equals 4.
Red is a primary colour.
3 answers in 0.71s
```

The 3 `ainvoke` calls run at the same time, so they take about as long as the slowest one — not the sum of all three, exactly like `gather_speed`.

Difference from Approach 1, and between Approaches 2–5: Approach 1's `ask()` has no timeout and is only ever called from a script's own `asyncio.run()`. Approach 2 adds the timeout and shows what a caller does with it in that same script-style setup. Approach 3 keeps the exact same `ask()` function, unmodified, and shows the *other* real shape it'll be called in — from inside a framework's already-running event loop, where `asyncio.run()` would be the wrong (and broken) thing to write. Approach 4 adds the two other async keywords — `async with` to open and close the client, `async for` to read a streamed reply. Approach 5 shows the same ideas on a LangChain chain: `ainvoke`, `astream`, and `gather` over several chain calls.

Which one should you actually use? Ship Approach 2's client-level timeout on every `AsyncOpenAI()` (or `OpenAI()`) client you create — it's one keyword argument, set once, that protects every call made through that client for the rest of the program's life. Approach 3's shape — `ask()` staying a plain, reusable coroutine with no `asyncio.run()` inside it — is exactly what this conversion needs to already look like before Doc12, since that's the literal function you'll import into a FastAPI route; writing it that way now means Doc12 is a five-minute wiring job instead of a second conversion. Use `async with` for any client or connection you open and want closed (Approach 4) — the MCP projects open every server session this way — and a chain's `ainvoke` / `astream` (Approach 5) whenever your code is already async.